```
# ============================================================
# SPAM DETECTION SYSTEM USING MACHINE LEARNING AND NLP
# MCA MINI PROJECT | ANACONDA | JUPYTER NOTEBOOK
# ============================================================

import os
import warnings
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import ipywidgets as widgets

from IPython.display import display, HTML, clear_output
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report, confusion_matrix
)

warnings.filterwarnings("ignore")
sns.set_theme(style="whitegrid")

# 1. PROJECT HEADER
```

```
display(HTML("""
<div style="background:linear-gradient(120deg,#172554,#2563eb);
padding:30px;border-radius:15px;text-align:center;color:white;">
<h1 style="color:white;">SPAM DETECTION SYSTEM</h1>
<h3 style="color:#dbeafe;">Machine Learning & Natural Language Processing</h3>
<p>Intelligent SMS Classification | MCA Mini Project</p>
</div>
"""))

# 2. LOAD DATASET
if not os.path.exists("spam.csv"):
    raise FileNotFoundError(
        "Please put spam.csv in the same folder as this notebook."
    )

df = pd.read_csv("spam.csv", encoding="latin-1")

if {"v1", "v2"}.issubset(df.columns):
    df = df[["v1", "v2"]].copy()
    df.columns = ["label", "message"]
elif {"label", "message"}.issubset(df.columns):
    df = df[["label", "message"]].copy()
else:
    raise ValueError("Dataset must contain v1/v2 or label/message columns.")

# 3. DATA PREPROCESSING
df = df.dropna(subset=["label", "message"]).copy()
df["label"] = df["label"].astype(str).str.strip().str.lower()
df["message"] = df["message"].astype(str).str.strip()

df = df[df["label"].isin(["spam", "ham"])]
```

```
df = df[df["message"] != ""]
df = df.drop_duplicates()

# Remove conflicting labels for identical messages
conflicts = df.groupby("message")["label"].nunique()
conflicts = conflicts[conflicts > 1].index
df = df[~df["message"].isin(conflicts)].copy()

df["label"] = df["label"].map({"ham": 0, "spam": 1})
df.reset_index(drop=True, inplace=True)

print("\nDATASET INFORMATION")
print("=" * 50)
print("Total Messages:", len(df))
print("Spam Messages:", (df["label"] == 1).sum())
print("Ham Messages:", (df["label"] == 0).sum())
print("Missing Values:\n", df.isnull().sum())
display(df.head())

# 4. EXPLORATORY DATA ANALYSIS
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

counts = df["label"].value_counts().sort_index()

sns.barplot(
    x=["Ham", "Spam"],
    y=[counts.get(0, 0), counts.get(1, 0)],
    hue=["Ham", "Spam"],
    palette=["#2563eb", "#ef4444"],
    legend=False,
    ax=axes[0]
```

```
)
axes[0].set_title("Spam vs Ham Message Count", fontsize=14)
axes[0].set_ylabel("Number of Messages")

for container in axes[0].containers:
    axes[0].bar_label(container)

axes[1].pie(
    [counts.get(0, 0), counts.get(1, 0)],
    labels=["Ham", "Spam"],
    autopct="%1.1f%%",
    colors=["#3b82f6", "#f97316"],
    startangle=90,
    explode=(0, 0.08)
)
axes[1].set_title("Message Percentage Distribution")

plt.tight_layout()
plt.show()

# 5. MESSAGE LENGTH ANALYSIS
df["message_length"] = df["message"].str.len()

plt.figure(figsize=(10, 5))
sns.histplot(
    data=df,
    x="message_length",
    hue="label",
    bins=40,
    kde=True,
    palette={0: "#2563eb", 1: "#ef4444"}
```

```
)
plt.title("SMS Message Length Distribution", fontsize=15)
plt.xlabel("Message Length")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# 6. SPLIT DATA INTO TRAINING AND TESTING
X = df["message"]
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTRAINING MESSAGES:", len(X_train))
print("TESTING MESSAGES:", len(X_test))

# 7. DEFINE MACHINE LEARNING MODELS
models = {
    "Naive Bayes": MultinomialNB(),
    "Logistic Regression": LogisticRegression(
        max_iter=1000, class_weight="balanced"
    ),
    "Support Vector Machine": LinearSVC(
        class_weight="balanced", random_state=42
    )
}
```

```
trained_models = {}
results = []

# 8. TRAIN AND EVALUATE MODELS
for name, algorithm in models.items():

    pipeline = Pipeline([
        ("tfidf", TfidfVectorizer(
            lowercase=True,
            stop_words="english",
            ngram_range=(1, 2),
            max_features=30000,
            min_df=2,
            sublinear_tf=True
        )),
        ("classifier", algorithm)
    ])

    pipeline.fit(X_train, y_train)
    predictions = pipeline.predict(X_test)

    trained_models[name] = pipeline

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, predictions),
        "Precision": precision_score(
            y_test, predictions, zero_division=0
        ),
        "Recall": recall_score(
```

```
            y_test, predictions, zero_division=0
        ),
        "F1-Score": f1_score(
            y_test, predictions, zero_division=0
        )
    })

    print(name, "training completed!")

# 9. MODEL COMPARISON
results_df = pd.DataFrame(results)

print("\nMODEL PERFORMANCE COMPARISON")
display(
    results_df.style.format({
        "Accuracy": "{:.2%}",
        "Precision": "{:.2%}",
        "Recall": "{:.2%}",
        "F1-Score": "{:.2%}"
    }).highlight_max(
        subset=["Accuracy", "Precision", "Recall", "F1-Score"],
        color="#bbf7d0"
    )
)

# 10. PERFORMANCE GRAPH
results_df.set_index("Model")[
    ["Accuracy", "Precision", "Recall", "F1-Score"]
].plot(
    kind="bar",
    figsize=(12, 6),
```

```
    colormap="viridis",
    ylim=(0, 1.1)
)

plt.title("Machine Learning Model Comparison",
          fontsize=16, fontweight="bold")
plt.ylabel("Performance Score")
plt.xlabel("Algorithms")
plt.xticks(rotation=15)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()

# 11. SELECT BEST MODEL USING F1-SCORE
best_model_name = results_df.loc[
    results_df["F1-Score"].idxmax(), "Model"
]

best_model = trained_models[best_model_name]
y_pred = best_model.predict(X_test)

print("\nBEST MODEL:", best_model_name)

# 12. CLASSIFICATION REPORT
print("\nCLASSIFICATION REPORT")
print(classification_report(
    y_test,
    y_pred,
    labels=[0, 1],
    target_names=["Ham", "Spam"],
    zero_division=0
```

```
))

# 13. CONFUSION MATRIX
cm = confusion_matrix(y_test, y_pred, labels=[0, 1])

plt.figure(figsize=(7, 5))
sns.heatmap(
   cm,
   annot=True,
   fmt="d",
   cmap="Blues",
   xticklabels=["Ham", "Spam"],
   yticklabels=["Ham", "Spam"],
   annot_kws={"size": 15}
)
plt.title("Confusion Matrix - Best Model",
       fontsize=15, fontweight="bold")
plt.xlabel("Predicted Label")
plt.ylabel("Actual Label")
plt.tight_layout()
plt.show()

# 14. SAVE TRAINED MODEL
joblib.dump(best_model, "spam_detection_model.pkl")
print("\nModel saved as spam_detection_model.pkl")

# 15. CUSTOM MESSAGE PREDICTION FUNCTION
def detect_spam(message):

   if not isinstance(message, str) or not message.strip():
      print("Please enter a valid message.")
```

```
        return

    prediction = best_model.predict([message])[0]

    if prediction == 1:
        display(HTML("""
        <div style="background:#fee2e2;padding:20px;
        border-left:6px solid #dc2626;border-radius:10px;
        color:#991b1b;">
        <h2>SPAM MESSAGE</h2>
        <p>This message resembles spam examples in the dataset.</p>
        </div>
        """))
    else:
        display(HTML("""
        <div style="background:#dcfce7;padding:20px;
        border-left:6px solid #16a34a;border-radius:10px;
        color:#166534;">
        <h2>HAM MESSAGE</h2>
        <p>This message resembles legitimate examples in the dataset.</p>
        </div>
        """))

    print("Message:", message)
    print("Model:", best_model_name)
    print("Note: This is a prediction, not a security guarantee.")

# 16. SAMPLE TESTING
print("\nSAMPLE PREDICTIONS")

detect_spam(
```

```
    "Congratulations! You have won a free prize. Claim now!"
)

detect_spam(
    "Hello, your class will start tomorrow at 10 AM."
)

# 17. INTERACTIVE DETECTION DASHBOARD
message_input = widgets.Textarea(
    placeholder="Enter your SMS message here...",
    description="Message:",
    layout=widgets.Layout(width="90%", height="100px")
)

check_button = widgets.Button(
    description="Detect Spam",
    button_style="primary",
    icon="search",
    layout=widgets.Layout(width="180px", height="40px")
)

output = widgets.Output()

def check_message(button):
    with output:
        clear_output(wait=True)
        detect_spam(message_input.value)

check_button.on_click(check_message)

display(HTML("""
```

```
<div style="background:linear-gradient(120deg,#172554,#2563eb);
padding:20px;border-radius:12px;color:white;text-align:center;">
<h2 style="color:white;">SMS SPAM DETECTION DASHBOARD</h2>
<p>Enter a message to classify it as Spam or Ham</p>
</div>
"""))

display(message_input)
display(check_button)
display(output)

# 18. FINAL PROJECT SUMMARY
display(HTML(f"""
<div style="background:#eff6ff;padding:22px;
border-radius:12px;border:1px solid #bfdbfe;">
<h2 style="color:#1e40af;">PROJECT COMPLETED SUCCESSFULLY</h2>
<p><b>Total Messages:</b> {len(df)}</p>
<p><b>Training Samples:</b> {len(X_train)}</p>
<p><b>Testing Samples:</b> {len(X_test)}</p>
<p><b>Best Algorithm:</b> {best_model_name}</p>
<p><b>Test Accuracy:</b> {accuracy_score(y_test,y_pred):.2%}</p>
<p><b>Spam F1-Score:</b> {f1_score(y_test,y_pred):.2%}</p>
<p><b>Saved Model:</b> spam_detection_model.pkl</p>
</div>
"""))

print("\nTHANK YOU! SPAM DETECTION PROJECT FINISHED.")
```